

DATA ENGINEERING / MONGODB / CAMPUS LIFE

# Orbit – Campus Discovery

Campus Event Manager: NoSQL Project Report

Vamshikar Thummala

8 September 2026

Master 1 – Data Engineering and Cloud Computing, MCS DE1

Source repository: <https://github.com/GajjarVaidehi/NoSQL-Final-Project>

## 1 Project overview and architecture

Orbit is a campus event management application built for the NoSQL Development Project. It manages members, events and registrations through a browser interface backed by real MongoDB operations. The objective is to demonstrate document modelling, CRUD, embedded-array updates and aggregation while keeping the application small enough to install locally.

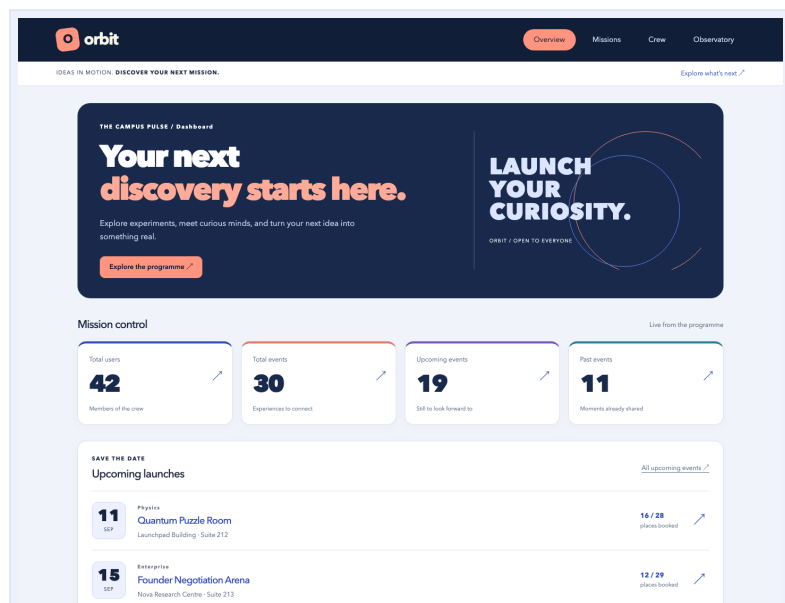
**Technology.** Python, FastAPI and PyMongo implement the backend; Jinja2 renders HTML on the server. Bootstrap and custom CSS provide a responsive interface, with small JavaScript interactions for confirmations. Pytest and FastAPI's TestClient validate the application. No ODM hides the database queries.

| Component                  | Responsibility                                                           |
|----------------------------|--------------------------------------------------------------------------|
| app/main.py                | Application setup, dashboard, static files, health and error handling.   |
| app/routes/                | Event CRUD and registration, users and history, six analytics functions. |
| app/database.py, config.py | Shared MongoClient, environment-based connection and database selection. |
| app/templates/, static/    | Jinja2 views and the Orbit visual design.                                |
| scripts/, tests/           | Schema/index initialization, seed generation and regression tests.       |

A browser request reaches a FastAPI route, which queries MongoDB through PyMongo and passes the result to a template. Form submissions validate input, update documents and normally redirect to the relevant page. The connection uses timezone-aware dates and a five-second server-selection timeout.

### 1.1 Dashboard – overview of campus activity

The home page uses `count_documents` for user/event counts and date filters for upcoming/past events. A projected query sorted by `startDate`, limited to five, supplies the next-events list. The numbers are read from MongoDB, not embedded in HTML.



**Figure 1:** Dashboard snapshot: 42 users, 30 events, 19 upcoming and 11 past events. The dashboard uses horizontal navigation and a navy mission-control banner.

## 2 NoSQL data model and consistency

The current development database is `orbit_campus`. It contains two collections, `users` and `events`. MongoDB assigns an `ObjectId` to each document. The brief specifies `campus_events`; this naming difference is recorded in Section 6 rather than concealed.

### 2.1 Document structure

```
users: {
  _id: ObjectId,
  firstName: String,
  lastName: String,
  email: String,
  department: String,
  role: "student" | "staff" | "organizer",
  interests: [String],
  createdAt: Date
}

events: {
  _id: ObjectId,
  title: String,
  description: String,
  category: String,
  tags: [String],
  startDate: Date,
  endDate: Date,
  capacity: Integer,

  location: {
    building: String,
    room: String,
    address: String
  },

  organizerId: ObjectId, // references users._id

  registrations: [{
    userId: ObjectId, // references users._id
    registeredAt: Date,
    status: "confirmed" | "cancelled" | "waitlisted"
  }],

  createdAt: Date,
  updatedAt: Date
}
```

This is a compact schema description, not an executable insertion script. Dates are stored as BSON dates, and references as BSON ObjectIds.

### 2.2 Embedding and referencing decisions

**1. Embed location and registrations.** A location belongs to an event and is usually read with it, so embedding avoids an unnecessary lookup. A registration contains event-specific facts: participant, booking time and status. Embedding this array makes the event the update boundary; registration changes can be atomic within one event document.

**2. Reference people instead of duplicating profiles.** An organizer or participant can appear in many events. The event stores `organizerId` and registrations store `userId`, while the person's name, email and department remain in `users`. Profile changes therefore do not require rewriting every event. Event details use `$lookup` for the organizer and retrieve participants by their referenced IDs.

**3. Keep interests and tags as arrays.** These are small multi-valued attributes used for display and filtering. A query such as `{tags: "Design"}` matches an element directly; tag analytics uses `$unwind`. There is no separate registration join collection.

### 2.3 Validation, indexes and safe updates

MongoDB JSON Schema validators enforce required fields, BSON types, positive capacity, an email pattern and controlled role/status values. Application validation rejects blank fields, malformed IDs and an end date that is not later than the start. A unique email index protects uniqueness even when requests overlap.

| Index                                                        | Purpose                                         |
|--------------------------------------------------------------|-------------------------------------------------|
| <code>users.email</code> (unique)                            | Prevent duplicate member emails.                |
| <code>events.startDate</code> ; <code>events.category</code> | Date ordering/filtering and category filtering. |
| <code>events.tags</code> ; <code>events.organizerId</code>   | Array-tag queries and organizer references.     |
| <code>events.(category, startDate)</code>                    | Compound category/date access pattern.          |

Registration insertion uses an atomic conditional `update_one`: the participant must not already occur in the array and its size must be below capacity.

`$addToSet` inserts the booking; `$pull` removes it. The user-ID predicate, rather than `$addToSet` alone, prevents duplicate people with different timestamps.

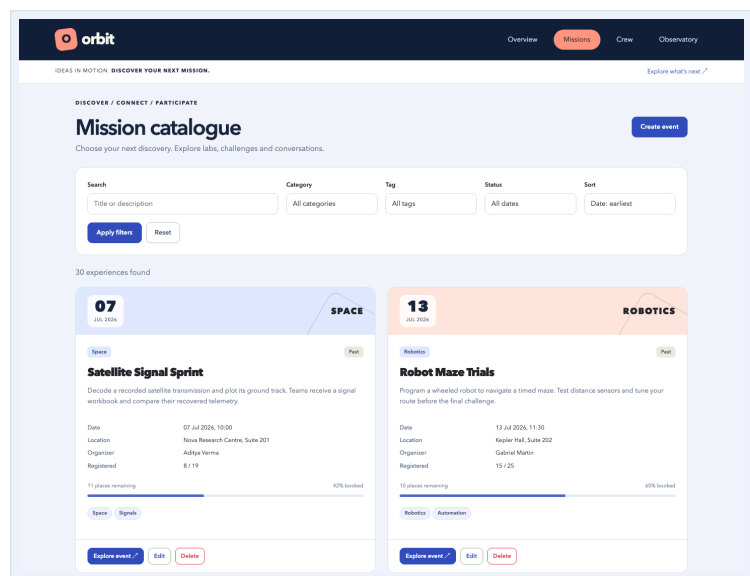
User deletion is blocked when an event references the user as organizer or participant. These checks do not constitute cross-collection transactions.

### 3 Application pages: events and registrations

#### 3.1 Events – browse, filter and manage

The programme page displays event titles, category, date, location, organizer, booked places, capacity and tags. Users can create, edit and delete events; combine title/description search with category, tag and date-status filters; and sort by date, title or capacity.

Search text is escaped before case-insensitive regular-expression matching. The form converts comma-separated tags into an array and builds the embedded location object.

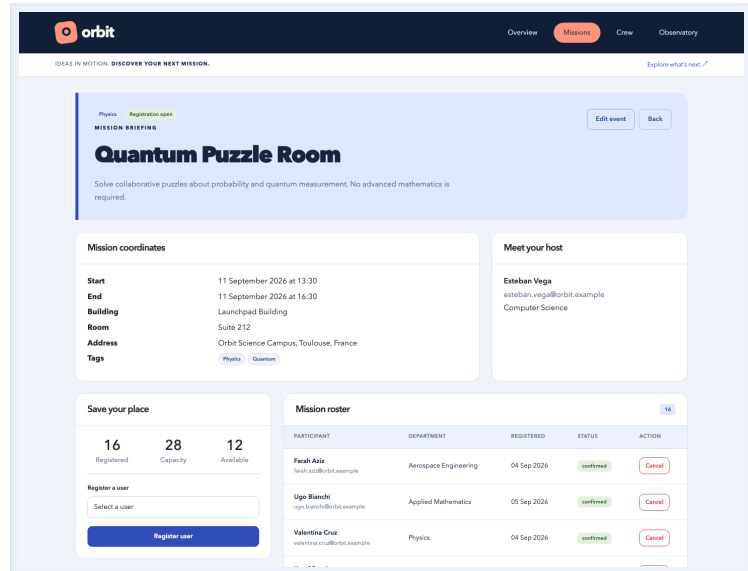


**Figure 2:** Event programme: filter controls and date-led cards. The visible cards contain real seeded events, with occupancy bars and remaining-place counts.

#### 3.2 Event details – participant management

A detail page retrieves one event, its organizer and referenced participants. It displays start/end dates, the complete location, tags and registration information.

The booking form selects an existing user. Duplicate or full-event requests return an explanatory message; successful changes redirect to the detail page with the refreshed participant list. Cancellation removes the embedded registration.



**Figure 3:** Event detail snapshot for the Quantum Puzzle Room. The page includes host information and, farther down, the registration summary and participant controls.

**Consistency example.** If capacity is one, the first booking succeeds; a second booking is refused, and booking the same participant again leaves only one registration.

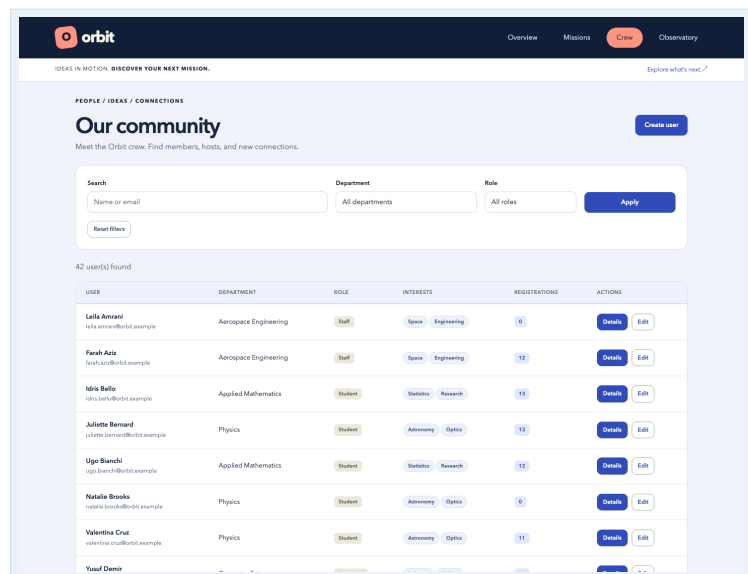
Editing cannot reduce capacity below the current registration count. Event deletion removes that event together with its embedded registrations.

## 4 Application pages: users and analytics

### 4.1 Users – directory and participation history

The community directory supports name/email search, department and role filters, and member creation/editing. It shows interests and registration counts.

A linked member detail view lists registered events, registration status, and upcoming/past participation counts; members with no bookings still have a valid detail page. A lookup between users and event registrations supplies participation information.



**Figure 4:** Orbit crew directory with searchable member profiles and registration information. User deletion is blocked while references remain.

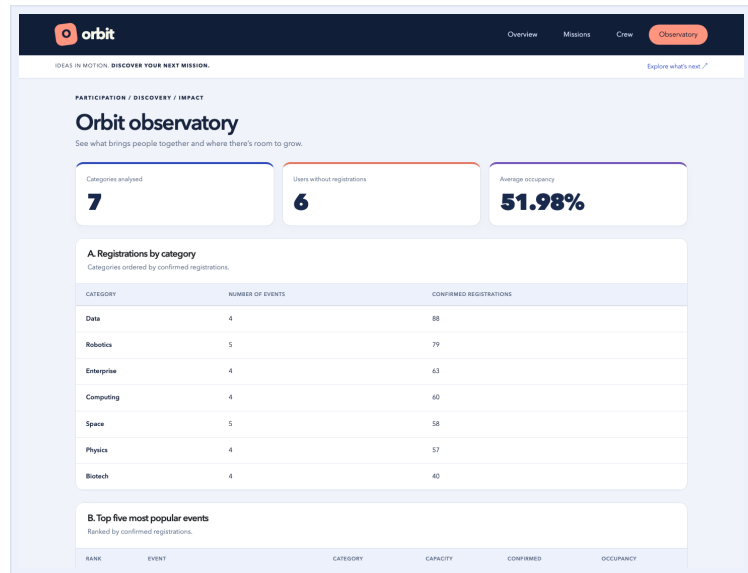
### 4.2 Analytics – six MongoDB analyses

The Insights page executes aggregation pipelines from `app/routes/analytics.py`.

It presents:

- (A) confirmed registrations by category,
- (B) the five most popular events and occupancy,
- (C) users without registrations,
- (D) events above average occupancy,
- (E) tag frequency, and
- (F) monthly event and registration totals.

These calculations are performed from stored documents.



**Figure 5:** Upper portion of Orbit Observatory. Additional tables for the remaining analyses are available by scrolling.

The category and popularity analyses count only registrations whose status is **confirmed**. Monthly totals count all embedded registrations.

These totals coincide in the seed because all 445 inserted registrations are confirmed; the distinction matters if other allowed statuses are stored later.

## 5 Aggregation focus and dataset results

### 5.1 Pipeline 1: users with no registrations

**Question:** Which members have not joined any event?

The pipeline starts from **users**, preserving the members whose absence from events is the subject of the query.

```
$lookup: {
  from: "events",
  localField: "_id",
  foreignField: "registrations.userId",
  as: "registeredEvents"
}

$match: {
  registeredEvents: { $size: 0 }
}

$project: {
  _id: 1,
  firstName: 1,
  lastName: 1,
  email: 1,
  department: 1,
  role: 1
}

$sort: {
  lastName: 1,
  firstName: 1
}
```

The lookup traverses references inside an embedded array. A user with no matching event receives an empty `registeredEvents` array; `$match` retains exactly those users.

Projection removes the joined payload, and sorting creates a readable directory. Starting from events and unwinding registrations would lose unregistered users.

The seed deliberately reserves six members without bookings, giving this analysis a non-empty result.

## 5.2 Pipeline 2: events above average occupancy

**Question:** Which events attract a higher proportion of their available places than the average event?

For event  $i$ , with confirmed count  $r_i$  and positive capacity  $c_i$ :

$$o_i = 100 \frac{r_i}{c_i}, \quad \bar{o} = \frac{1}{N} \sum_{i=1}^N o_i, \quad \text{retain events where } o_i > \bar{o}.$$

1. `$project` computes the confirmed count using `$filter` followed by `$size`, retaining event identity and capacity.
2. A second projection calculates occupancy using `$divide` and `$multiply`.
3. `$group with _id: null` computes `$avg` and retains event documents through `$push: "$$ROOT"`.
4. `$unwind` restores one row per event while retaining the shared average.
5. `$match with $expr` compares each event's occupancy against that average.
6. Final projection rounds output to two decimals; sorting orders occupancy descending, then title.

The mean is an *unweighted average of event percentages*, not total bookings divided by total capacity.

Comparison occurs before rounding. The seed produces an average of **51.98%**, with **16 events above average**.

For large datasets, retaining every event in one grouped document would need a more scalable design.

## 5.3 Reproducible data and interpretation

|                         |     |                               |    |
|-------------------------|-----|-------------------------------|----|
| Users                   | 42  | Events                        | 30 |
| Categories              | 7   | Distinct tags                 | 37 |
| Confirmed registrations | 445 | Members without registrations | 6  |
| Full events             | 3   | Events without bookings       | 3  |
| Past events at seeding  | 11  | Future events at seeding      | 19 |

The seed uses relative day offsets, varied durations, capacities and locations; all organizer and participant references resolve to generated users.

Registration timestamps precede their events. Re-running the seed recreates the configured database's two project collections' contents, so figures are reproducible while calendar dates move with the run date.

For example, *Venture Demo Exchange* leads by count with 28 confirmed bookings out of 42 places (66.67%).

Popularity by count is therefore different from occupancy: a smaller event can be full with fewer bookings.

The Data category leads category registrations with 88 bookings. These figures were cross-checked from the seed definitions and describe the demonstration dataset, not measured campus usage.

# 6 Verification, limitations and conclusion

## 6.1 Verification evidence

The development validation run on 8 September 2026 completed with **19 tests passed**.

The suite was rerun against the separate Orbit database after the redesign.

Test sources are:

`tests/test_pages.py`

and

`tests/test_redesign.py`.

| Area              | Coverage                                                                                                         |
|-------------------|------------------------------------------------------------------------------------------------------------------|
| Pages and errors  | Health, dashboard, lists, forms, analytics, static CSS, malformed IDs and unknown routes.                        |
| Data integrity    | Counts, unique emails, valid references, chronology, capacities, full/empty events and members without bookings. |
| Detail views      | Detail and edit pages for every seeded event and user.                                                           |
| Management flow   | User/event creation, editing and deletion; duplicate email rejection; referenced-user deletion protection.       |
| Registration flow | Successful booking, duplicate refusal, full-event refusal and cancellation.                                      |
| Filtering         | Event search, category/tag/status filters and empty event/user search results.                                   |

Headless Chrome screenshots were captured at desktop and mobile widths for visual review.

Screenshots cover the dashboard, events, event detail, community, insights and event form.

Python compilation and Git whitespace checks also passed.

These checks support functional confidence, but do not prove complete compliance with every sentence of the assignment or safety under all concurrent operations.

## 6.2 Reproduction

From the repository root, with Python and MongoDB available:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python -m scripts.initialize_database
python -m scripts.seed_database
uvicorn app.main:app --reload

# In another activated terminal,
# with MongoDB running:

python -m pytest -v
```

The standard local URL is:

`http://127.0.0.1:8000`

Configure `MONGODB_URI` and `MONGODB_DATABASE` in `.env`; keep private credentials outside version control.

Seeding replaces the users/events contents of the selected database and should be run only against the intended demonstration database.

## 6.3 Current limitations and remaining brief alignment

The checked-in design uses the Orbit identity and `orbit_campus`, whereas the assignment specifies Campus Event Manager and `campus_events`.

The current dashboard shows a past-event KPI but lacks the requested total-registration KPI and most-popular-event summary.

Event cards show occupancy/remaining places but no explicit Full badge; event details show counts but no occupancy percentage.

These are remaining alignment tasks, not features demonstrated by the screenshots.

Analytics filters confirmed statuses, but several operational counts and capacity checks use the full registration-array length.

This is consistent with the all-confirmed seed and removal-based cancellation; stored waitlisted/cancelled entries would require status-aware counting throughout.

The analytics header also derives its average from above-average results, so it can show zero when all event occupancies are equal.

Cross-collection deletion/registration races and high-volume performance are not covered by the present tests.

## 6.4 Conclusion

The main engineering challenges are balancing embedding with reusable user references, maintaining capacity and duplicate-booking constraints, and calculating comparable indicators from arrays.

The project addresses these through a two-collection model, conditional single-document updates and explicit aggregation pipelines.

The next priorities are the brief-alignment items above, consistent status semantics, pagination and stronger concurrent-operation tests.

Authentication and deployment can be added later but are outside the required project scope.

**Report basis:** *NoSQL Development Project – Campus Event Manager*, MCS DE1, released 1 September 2026, especially Section 9 (report requirements); local application source; seed definitions; and the 8 September 2026 verification screenshots.